

MODELO DE DADOS

Documentação de Banco de Dados

Estrutura, relacionamentos, integridade e proteção dos dados

Projeto	BemEstarClinic
SGBDs	PostgreSQL (sistema de gestão) SQLite (site e painel de conteúdo)
Entidades	22 tabelas — 15 no PostgreSQL, 7 no SQLite
Migrations	4 aplicadas, versionadas em migrations/
Situação	Em produção
Data da análise	12 de agosto de 2026

Sumário

1. Visão geral	3
1.1 Dois bancos, dois motores	3
1.2 Como se chegou aqui	3
1.3 Estratégia de persistência	3
2. Banco do site — SQLite	4
2.1 Privacidade no contador de visitas	4
3. Banco da gestão — PostgreSQL	5
3.1 Acesso e operação	5
3.2 Tabelas de sistema e apoio	5
3.3 pacientes	6
3.4 atendimentos (agenda)	6
3.5 prontuario (a pasta)	7
3.6 prontuario_registros (os lançamentos)	7
3.7 anamneses	7
3.8 historico e auditoria	8
4. Diagrama entidade-relacionamento	9
5. Relacionamentos	10
5.1 Um para muitos (1:N)	10
5.2 Um para um (1:1)	10
5.3 Relacionamento polimórfico	10
5.4 Não há N:N	10
6. Integridade	11
6.1 Chaves primárias	11
6.2 Índices únicos — as regras gravadas no banco	11
6.3 Índices de desempenho	11
6.4 Restrições NOT NULL	11
6.5 CHECK constraints	12
6.6 Chaves estrangeiras — a ausência mais relevante do modelo	13
7. Migrations	14
7.1 Qualidade das migrations	14
8. Dados iniciais (seeds)	15
8.1 Cores oficiais dos procedimentos	15
8.2 Sequências de numeração	15

9. Proteção dos dados	16
9.1 Colunas cifradas	16
9.2 O que NÃO é cifrado — e por quê	16
9.3 Busca sobre campo cifrado	16
9.4 Backup e restauração	17
9.5 Superfície de exposição do banco	17
10. Consultas relevantes	18
11. Observações e recomendações	19
11.1 Problemas encontrados	19
11.2 Recomendações	19

1. Visão geral

1.1 Dois bancos, dois motores

O sistema usa **dois bancos de dados distintos**, com motores diferentes, no mesmo processo. Eles não se comunicam: não há consulta, chave estrangeira ou junção entre eles.

Banco	Motor	Onde vive	Guarda	Tabelas
Site e painel	SQLite	data/site.db (arquivo)	Conteúdo editável do site e contagem de visitas	7
Sistema de gestão	PostgreSQL 16	Base bemestar_gestao, apenas em 127.0.0.1	Pacientes, agenda, anamneses, prontuário, auditoria	15

1.2 Como se chegou aqui

Até a versão 1.19.0 (28/07/2026) **os dois bancos eram SQLite**. A gestão migrou para PostgreSQL numa virada única, executada pelo script `migrar-dados.js`, que copiou tabela por tabela dentro de **uma única transação**, preservou os ids, acertou os contadores de sequência e conferiu contagem e amostra de conteúdo nos dois lados.

FATO

A recomendação técnica anterior, registrada no projeto, era **manter o SQLite**: a escala real (dois a três profissionais, cerca de 9 mil atendimentos por ano, banco de 76 KB) é irrisória para ele, e os dois riscos apontados não eram do motor — eram a ausência de backup automático e o fato de o driver `node:sqlite` ser experimental. Ambos foram resolvidos antes da migração. A troca por PostgreSQL, portanto, não foi motivada por limite de capacidade.

INFERÊNCIA

Os ganhos que a migração de fato trouxe, visíveis no código: **índices únicos que realmente nascem** (no SQLite eles eram criados dentro de `try/catch` e podiam simplesmente não existir), **esquema versionado** em migrations transacionais, e um caminho de backup por `pg_dump` restaurável em qualquer servidor.

1.3 Estratégia de persistência

Decisão	Como está	Por quê
Liga/desliga como INTEGER	ativo, arquivado, consentimento, primeira, encaixe etc. são 0 ou 1, e não BOOLEAN	Virar boolean obrigaria a rever cada <code>ativo=1</code> do SQL, cada JSON que a tela envia e cada <code>if (x.ativo)</code> do JavaScript. O ganho seria estético; o risco, real
Datas como TEXT em ISO	'2026-07-28' ou '2026-07-28T13:05:00Z', e não DATE/TIMESTAMP	É como o sistema já grava, compara e ordena — e a ordenação alfabética de ISO é a cronológica. Converter mudaria o que a API devolve para a tela (o driver entregaria objeto <code>Date</code> no lugar da string) e quebraria as impressões
Respostas de anamnese em JSON	Coluna <code>anamneses.dados</code> guarda texto JSON	Acrescentar pergunta ao modelo não exige migração de esquema
Chaves substitutas	Todas as tabelas usam <code>id INTEGER GENERATED BY DEFAULT AS IDENTITY</code>	Preserva os ids vindos do SQLite e permite INSERT com id explícito na migração

As duas primeiras decisões estão comentadas na própria migration inicial, com a observação de que migrar esses tipos, quando fizer sentido, deve ser um passo próprio e testado — não “de carona” numa troca de banco.

2. Banco do site — SQLite

Criado e mantido pelo próprio `server.js`, com `CREATE TABLE IF NOT EXISTS` no boot. Não usa migrations.

Tabela	Objetivo	Colunas	Chaves e índices
settings	Todo o conteúdo textual do site, em chave/valor. Guarda os 84 campos editáveis, o hash da senha do painel, o sal do contador de visitas e o estado do modo manutenção	key TEXT, value TEXT NOT NULL	PK: key
services	Especialidades, cada uma com página própria	id, title NOT NULL, text, sort, slug, content	PK: id
portfolio	Fotos da seção Nosso Espaço	id, title NOT NULL, subtitle, image, sort	PK: id
testimonials	Depoimentos	id, text NOT NULL, name, role, initials, sort	PK: id
team	Profissionais exibidos no site	id, name NOT NULL, role, bio, photo, sort, whatsapp, especialidades, na_home	PK: id
posts	Matérias do blog	id, title NOT NULL, slug NOT NULL UNIQUE , excerpt, content, image, date, sort	PK: id · UNIQUE: slug
visits	Contagem de visitas ao site público	id, ip_hash NOT NULL, path, referrer, ua, day NOT NULL, ts NOT NULL	PK: id · idx(ip_hash, ts) · idx(day) · idx(ts)

PONTO DE ATENÇÃO - dívida técnica no banco do site

As colunas em destaque (`slug`, `content`, `whatsapp`, `especialidades`, `na_home`) **não estão no CREATE TABLE**: são acrescentadas por `ALTER TABLE` dentro de `try {} catch {}` vazio, logo depois. É exatamente o padrão que a gestão abandonou ao adotar migrations, e pelo mesmo motivo: o `catch` vazio engole “coluna já existe” e um erro de digitação, com o mesmo silêncio.

2.1 Privacidade no contador de visitas

A tabela `visits` **nunca guarda IP em texto claro**. Grava-se `sha256(sal + ip)`, com `sal` aleatório gerado por instalação e persistido em `settings.visit_salt`.

- Sem o `sal`, o hash de um IPv4 seria quebrável por força bruta — são apenas 4 bilhões de endereços possíveis.
- Uma visita por hash a cada 30 minutos; robôs são descartados antes de gravar.
- **Retenção de 12 meses**, com limpeza automática — a LGPD exige prazo definido, não “para sempre”.
- `referrer` é gravado vazio quando a origem é o próprio site.

PONTO DE ATENÇÃO - ação pendente

O `visit_salt` desta instalação esteve exposto em repositório público, junto com uma cópia do `site.db` no histórico de dois commits. Enquanto ele não for rotacionado, os hashes já gravados não têm a proteção que a política de privacidade promete. Rotacionar o `sal` inutiliza a deduplicação dos registros antigos — o que, neste caso, é o efeito desejado.

3. Banco da gestão — PostgreSQL

Quinze tabelas. Estrutura definida em migrations/*.sql e aplicada uma única vez cada, com registro em schema_migrations.

3.1 Acesso e operação

Aspecto	Configuração
Base	bemestar_gestao, codificação UTF-8, criada com TEMPLATE template0 (garante a codificação independente do locale do servidor)
Usuário da aplicação	bemestar, dono da base
Escuta	Apenas 127.0.0.1. A porta 5432 não é aberta no firewall
Pool	Máximo de 10 conexões, ocioso encerrado em 30 s, conexão em até 8 s
Identificação	application_name = bemestar-restrito — aparece em pg_stat_activity e permite ver quem segura conexão
Transações	Q.tx(), com o cliente propagado por contexto assíncrono para que toda consulta interna use a mesma conexão

3.2 Tabelas de sistema e apoio

Tabela	Objetivo	Colunas	Restrições
schema_migrations	Controle das migrations aplicadas	versao TEXT PK · aplicada TIMESTAMPTZ NOT NULL DEFAULT now() · ms INTEGER	PK: versao
g_usuarios	Contas de acesso ao sistema	id · nome NOT NULL · email UNIQUE · senha_hash NOT NULL · perfil NOT NULL DEFAULT 'admin' · ativo DEFAULT 1 · profissional_id · criado	PK: id · UNIQUE: email
g_config	Configurações internas e contadores de numeração (pront_seq_2026, pac_seq_2026) e travas de sementeira	key TEXT PK · value TEXT	PK: key
convenios	Formas de pagamento aceitas	id · nome NOT NULL · registro · contato · observacao · ativo DEFAULT 1 · sort DEFAULT 0 · criado	PK: id
salas	Consultórios	id · nome NOT NULL · ativo DEFAULT 1 · sort DEFAULT 0 · criado	PK: id
procedimentos	O que se agenda	id · nome NOT NULL · tipo DEFAULT 'Consulta' · valor · duracao INTEGER DEFAULT 40 · cor · anamnese_modelo · ativo DEFAULT 1 · sort DEFAULT 0 · criado	PK: id
profissionais	Quem atende	id · nome NOT NULL · especialidade (JSON) · registro · contato · cor · ativo DEFAULT 1 · criado	PK: id
documentos_gestao	Anexos por paciente	id · paciente_id · tipo · titulo · arquivo · data · criado	PK: id

FATO

procedimentos.nome **repete de propósito**: “Ozonioterapia” existe duas vezes, uma como Consulta e outra como Sessão. O que identifica o *tratamento* é o nome, não o id da linha — e é o nome que vira a chave do prontuário. profissionais.especialidade guarda um **array JSON** com os rótulos “Nome (Tipo)” dos procedimentos que aquele profissional realiza; lista vazia significa “não restringe”.

3.3 pacientes

A maior tabela do sistema: **50 colunas**, espelhando a ficha de cadastro usada pela recepção. Agrupadas por natureza:

Grupo	Colunas
Identificação	codigo (PAC-AAAA-00000, gerado pelo servidor) · nome NOT NULL · nome_contato · foto · juridica DEFAULT 0 · estrangeiro DEFAULT 0
Documentos	cpf · rg
Pessoais	sexo · nascimento · naturalidade · estado_civil · religiao · profissao · escolaridade · mae · pai
Clínicos de cadastro	altura · peso · cor_pele · sangue · prioridade
Endereço	cep · endereco · numero · bairro · cidade · complemento
Contato	celular · telefone · email · canal · avisos DEFAULT 1
Responsável (menor de idade)	resp_nome · resp_cpf · resp_rg · resp_nascimento
Relacionamento	convenio_id · tag · indicacao · observacao · consentimento DEFAULT 0
Ciclo de vida	criado · ativo DEFAULT 1 · inativo_em · inativo_motivo · reativado_em · arquivado NOT NULL DEFAULT 0 · arquivado_em

FATO

ativo e arquivado são colunas **diferentes e independentes**, e a migration 004 documenta o porquê dentro do próprio banco, com COMMENT ON COLUMN: "Organização da tela: 1 some da lista. NÃO significa inativo — para isso existe ativo."

3.4 atendimentos (agenda)

Grupo	Colunas
Vínculos	paciente_id · profissional_id · sala_id · convenio_id · procedimento_id · especialidade · prontuario_id
Identificação na agenda	nome_agenda · celular
Horário	data · hora · hora_fim
Comercial	valor · convenio_id
Marcadores	primeira DEFAULT 0 · encaixe DEFAULT 0 · lembrete DEFAULT 1 · nps DEFAULT 1
Situação	status DEFAULT 'Agendado' · observacoes · criado

FATO

prontuario_id começa nulo. **O agendamento não cria prontuário**: ele só se liga a uma pasta que já exista para aquele paciente naquele procedimento. O vínculo é preenchido quando a anamnese é finalizada, ou manualmente. A coluna não está na whitelist de escrita da API — quem a preenche é sempre o servidor.

PONTO DE ATENÇÃO - colunas sem consumidor

As colunas lembrete e nps existem, têm valor padrão 1 e são marcáveis no formulário, mas **nenhuma rotina do sistema as consome**. Hoje são apenas intenção registrada.

3.5 prontuario (a pasta)

Coluna	Tipo	Obrig.	Papel
id	INTEGER IDENTITY	PK	Identificador interno
numero	TEXT	não	PR-AAAA-00000. Gerado pelo servidor, nunca editável
paciente_id	INTEGER	NOT NULL	De quem é a pasta
especialidade	TEXT	NOT NULL	Guarda o nome do procedimento , apesar do nome da coluna
profissional	TEXT	não	Nome digitado — apenas informativo
profissional_id	INTEGER	não	O dono da pasta. É por ele que funciona o recorte do perfil profissional
status	TEXT DEFAULT 'Ativo'	não	Ativo ou Alta
aberto_em / alta_em / alta_motivo / reativado_em	TEXT	não	Marcos do acompanhamento
observacao	TEXT	não	Texto livre da pasta
usuario_id	INTEGER	não	Quem <i>criou</i> — pode ser a recepção
criado	TEXT	não	Data de cadastro
arquivado / arquivado_em	INTEGER NOT NULL DEFAULT 0 / TEXT	não	Organização de tela

FATO

A migration 003 explica por que `profissional_id` precisou existir. `profissional` (texto) é frágil: muda a grafia, entra um “Dr.”, e o registro escapa do filtro. E `usuario_id` erra nos dois sentidos — se a recepção abre a pasta, o profissional deixa de ver o próprio prontuário; se um profissional cadastra para outro, passa a ver o que não é dele. **É registro de autoria, não de responsabilidade.**

3.6 prontuario_registros (os lançamentos)

Cada avaliação, evolução, plano terapêutico ou encaminhamento é **uma linha datada**. Nada é excluído — apenas arquivado.

Coluna	Papel
prontuario_id NOT NULL	A pasta a que pertence
tipo NOT NULL	avaliacao · evolucao · plano · encaminhamento
texto	O registro clínico (aceita formatação; passa por lista de tags permitidas)
data	Data do lançamento, preenchida automaticamente
profissional	Quem assina
usuario_id	O autor. É por ele que o lançamento fica privado, mesmo dentro de uma pasta compartilhada
anexo	Arquivo ligado ao lançamento
arquivado / arquivado_em	Some da tela, permanece no banco
criado / atualizado	atualizado alimenta o aviso “editado em...” na lista

3.7 anamneses

Coluna	Papel
paciente_id NOT NULL	De quem é
tipo NOT NULL	psicanalise · ozonio · integrativas

Coluna	Papel
dados	Respostas em JSON. Acrescentar pergunta ao modelo não altera o esquema
procedimento	Procedimento que motivou a anamnese
status DEFAULT 'Rascunho'	Rascunho ou Finalizada
finalizada_em	Quando foi fechada
prontuario_id	A pasta que ela abriu (ou reaproveitou)
profissional / profissional_id	Nome digitado / o dono , usado no recorte
data · usuario_id · criado · atualizado	Marcos e autoria

3.8 historico e auditoria

Duas trilhas com propósitos distintos, e vale não confundi-las:

Tabela	Responde	Escopo	Quem lê
historico	“O que aconteceu com este paciente / esta pasta?”	Eventos de negócio: abertura, alta, reativação, edição de lançamento com o trecho alterado. Colunas: entidade NOT NULL · entidade_id NOT NULL · evento NOT NULL · detalhe · usuario_id · usuario_nome · criado	Quem tem acesso à pasta
auditoria	“Quem mexeu no sistema, e quando?”	Eventos de operação: login, logout, tela aberta, criação, edição, exclusão. Colunas: criado NOT NULL · ip · usuario_id · usuario_nome · perfil · acao NOT NULL · modulo · entidade_id · resumo · detalhe · rota · metodo	Somente admin

FATO

Em auditoria, **resumo** e **detalhe** são cifrados, porque carregam o conteúdo do que foi feito: o nome do paciente cadastrado, o CPF corrigido, o trecho da evolução alterada. Sem isso a auditoria viraria a porta dos fundos do banco. O que fica legível é justamente o que serve para filtrar e agrupar *sem* revelar paciente: data, IP, autor, perfil, ação e módulo — e é por essas colunas que os índices existem.

4. Diagrama entidade-relacionamento

Relacionamentos do banco da gestão. **Todos são lógicos:** nenhuma chave estrangeira está declarada no banco (ver capítulo 6).

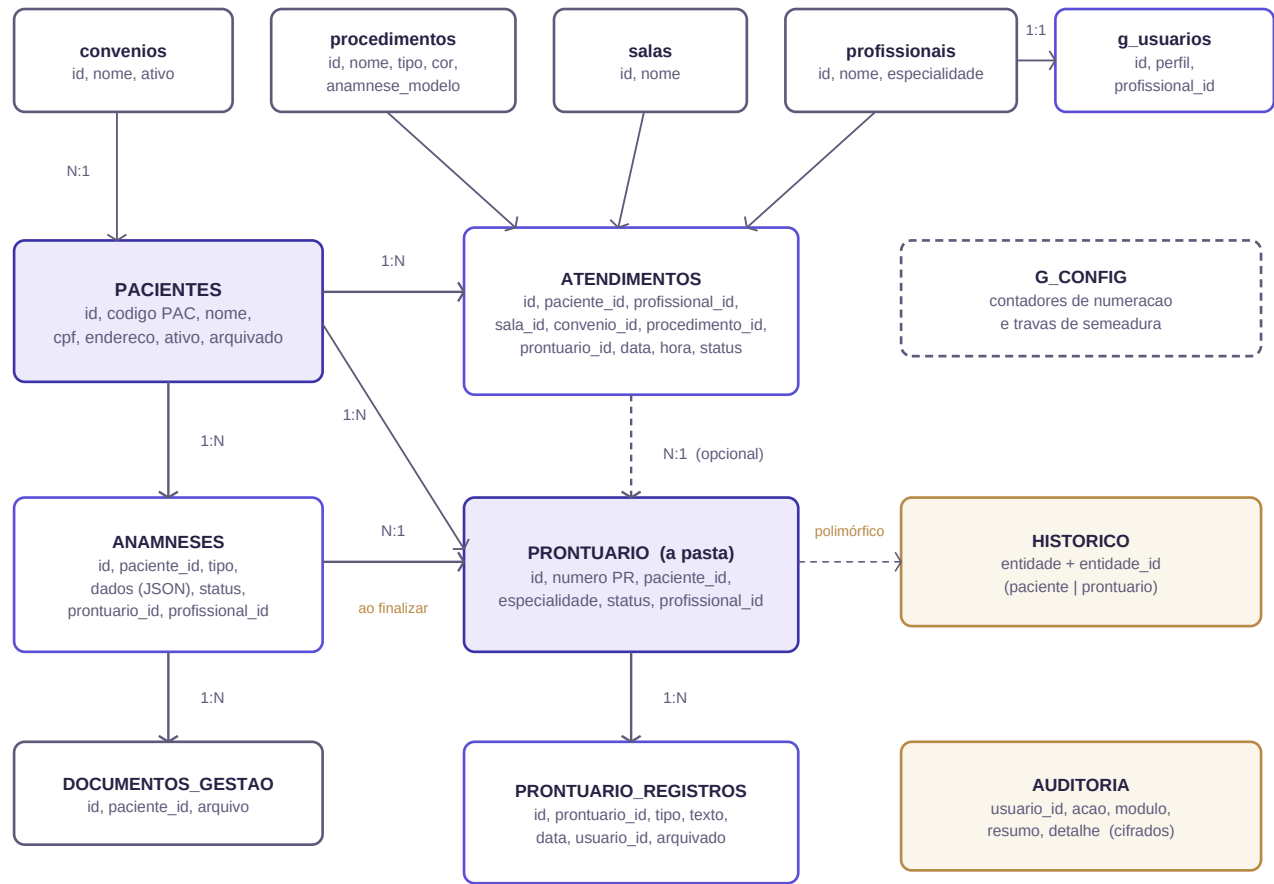


Figura 1 — Modelo entidade-relacionamento do sistema de gestão. Linhas tracejadas são vínculos opcionais ou polimórficos. Nenhuma delas existe como FOREIGN KEY no banco.

5. Relacionamentos

5.1 Um para muitos (1:N)

Origem	Destino	Coluna	Obrigatório?	Observação
pacientes	atendimentos	paciente_id	sim, por regra	Só se agenda para paciente cadastrado
pacientes	anamneses	paciente_id	NOT NULL	—
pacientes	prontuario	paciente_id	NOT NULL	Um por procedimento
pacientes	documentos_gestao	paciente_id	não	—
prontuario	prontuario_registros	prontuario_id	NOT NULL	O conteúdo clínico da pasta
prontuario	anamneses	prontuario_id	não	Nulo enquanto a anamnese é rascunho
prontuario	atendimentos	prontuario_id	não	Nulo até haver pasta para o par paciente + procedimento
profissionais	atendimentos	profissional_id	sim, por regra	Para quem foi marcado
profissionais	prontuario	profissional_id	não	O dono — base do recorte
profissionais	anamneses	profissional_id	não	O dono — base do recorte
procedimentos	atendimentos	procedimento_id	não	—
salas	atendimentos	sala_id	não	Usado na validação de choque
convenios	atendimentos / pacientes	convenio_id	não	—
g_usuarios	prontuario_registros	usuario_id	não	O autor do lançamento

5.2 Um para um (1:1)

`g_usuarios.profissional_id` -> `profissionais.id`. É o que liga a conta de acesso ao cadastro do profissional e permite que ele veja “a sua agenda”. Um profissional sem esse vínculo **não vê nada** das tabelas recortadas — o código usa -1 como dono, que nunca casa com um id real.

5.3 Relacionamento polimórfico

`historico` aponta para duas entidades por meio do par entidade (texto: paciente ou prontuario) + `entidade_id`. É o padrão que o índice `idx_hist_ent(entidade, entidade_id)` atende.

INFERÊNCIA

Esse desenho é o motivo técnico pelo qual `historico` jamais poderia ter chave estrangeira: uma FK aponta para **uma** tabela, e essa coluna aponta para duas. É uma justificativa válida **para esta tabela** — mas não explica a ausência de FK nas outras treze.

5.4 Não há N:N

Nenhuma tabela de junção existe no modelo. A relação que *seria* N:N — quais procedimentos cada profissional realiza — foi resolvida com um **array JSON** na coluna `profissionais.especialidade`, guardando os rótulos “Nome (Tipo)”.

PONTO DE ATENÇÃO - fragilidade do vínculo por texto

Como a lista guarda **rótulos de texto** e não ids, renomear um procedimento **quebra silenciosamente** a associação: o profissional deixa de “realizar” aquele procedimento e ele some do seletor de agendamento, sem erro e sem aviso. Esse mecanismo já gerou um chamado real — descrito no documento de Produto, item 7.1 — ainda que naquele caso a causa tenha sido lista desatualizada, e não renomeação.

6. Integridade

6.1 Chaves primárias

Todas as tabelas têm chave primária. Treze usam `INTEGER GENERATED BY DEFAULT AS IDENTITY`; `g_config` usa `key` e `schema_migrations` usa `versao`. O `BY DEFAULT` (em vez de `ALWAYS`) foi escolhido porque a migração precisava inserir com `id` explícito para preservar os `ids` vindos do SQLite.

6.2 Índices únicos — as regras gravadas no banco

Índice	Sobre	O que garante
<code>idx_pront_numero</code>	<code>prontuario(numero)</code> <code>WHERE numero IS NOT NULL</code>	Nem um backup restaurado por cima duplica número de prontuário
<code>idx_pront_pac_esp</code>	<code>prontuario(paciente_id, especialidade)</code>	A regra da clínica: um prontuário por paciente + procedimento. Mesmo que a tela falhe, o banco recusa o segundo
<code>idx_pac_codigo</code>	<code>pacientes(codigo)</code> <code>WHERE codigo IS NOT NULL AND codigo <> ''</code>	O código do paciente não repete
<code>g_usuarios.email</code>	UNIQUE de coluna	Login não duplicado
<code>posts.slug</code>	UNIQUE de coluna (SQLite)	URL de matéria única

FATO

Os índices parciais são deliberados: um índice único simples sobre `numero` trataria vários `NULL` como conflito em alguns motores, e sobre `codigo` a condição também exclui a string vazia. A migração para PostgreSQL foi precedida de um diagnóstico (`migrar-dados.js --conferir`) justamente porque o **SQLite antigo aceitava essas duplicatas** — lá os índices únicos eram criados dentro de `try/catch` e podiam simplesmente não nascer.

6.3 Índices de desempenho

Índice	Coluna(s)	Consulta que atende
<code>idx_atend_data</code>	<code>atendimentos(data)</code>	Agenda do dia e filtro de período
<code>idx_atend_pac</code>	<code>atendimentos(paciente_id)</code>	Atendimentos de um paciente
<code>idx_pront_pac</code>	<code>prontuario(paciente_id)</code>	Pastas de um paciente
<code>idx_anam_pac</code>	<code>anamneses(paciente_id)</code>	Anamneses de um paciente
<code>idx_preg_pront</code>	<code>prontuario_registros(prontuario_id)</code>	Lançamentos de uma pasta
<code>idx_hist_ent</code>	<code>historico(entidade, entidade_id)</code>	Linha do tempo de uma entidade
<code>idx_pront_prof</code>	<code>prontuario(profissional_id)</code>	Recorte do profissional — sem ele, cada tela varreria a tabela inteira
<code>idx_anam_prof</code>	<code>anamneses(profissional_id)</code>	Idem, para anamneses
<code>idx_aud_criado</code>	<code>auditoria(criado DESC)</code>	A tela abre pelo mais recente
<code>idx_aud_usuario / acao / modulo</code>	<code>auditoria(...)</code>	Os três filtros da tela
<code>idx_pac_arquivado</code>	<code>pacientes(id) WHERE arquivado = 0</code>	Índice parcial: só as linhas que ficam à vista entram nele
<code>idx_pron_arquivado</code>	<code>prontuario(id) WHERE arquivado = 0</code>	Idem

6.4 Restrições NOT NULL

Aplicadas com parcimônia, e sempre no que é estrutural: `nome` nos cadastros; `paciente_id` e `especialidade` em `prontuario`; `prontuario_id` e `tipo` em `prontuario_registros`; `paciente_id` e `tipo` em

anamneses; entidade, entidade_id e evento em historico; criado e acao em auditoria; e os dois arquivado.

6.5 CHECK constraints

NÃO IDENTIFICADO

Não há nenhuma CHECK constraint no banco. Valores de domínio fechado — perfil (admin/secretaria/profissional), status do prontuário (Ativo/Alta), status da anamnese (Rascunho/Finalizada), tipo do lançamento (avaliacao/evolucao/plano/encaminhamento) e tipo do procedimento — são validados **apenas na aplicação**.

6.6 Chaves estrangeiras — a ausência mais relevante do modelo

PONTO DE ATENÇÃO

O esquema não declara nenhuma **FOREIGN KEY**. As quatorze relações do capítulo 5 existem exclusivamente como colunas *_id mantidas pela aplicação. O banco aceita, hoje, um atendimento apontando para um paciente_id inexistente.

A ausência é herança fiel do SQLite: naquele motor a verificação de chave estrangeira é **desligada por padrão** e precisa ser ativada por conexão. A migration 001 se descreve como “tradução fiel do que existia”, e traduziu também essa característica.

O que a aplicação faz no lugar

- `vinculosDe(tabela, id)` conta os vínculos antes de excluir; se houver, responde **409** listando-os e sugerindo bloquear, inativar ou dar alta.
- Uma **rotina no boot solta vínculos órfãos** de anamneses e atendimentos, registrando “vínculos órfãos soltos” no log.
- `GET anamneses/:id` devolve `_prontuario` **resolvido no servidor**, e a tela decide por ele — não pelo campo, que pode ter sobrado apontando para pasta apagada.
- A exclusão obedece a uma ordem definida: lançamentos -> pasta -> anamneses -> agendamentos -> paciente.

Por que isso importa: dois defeitos reais já causados

Defeito	O que aconteceu	Correção aplicada
Vínculo órfão (v1.18.0)	Três anamneses apontavam para uma pasta que não existia mais. A lista não achava a pasta e mostrava “sem prontuário”; o formulário via o campo preenchido e escondia o botão Excluir. Registro impossível de apagar, sem motivo visível ao usuário	Migração no boot que solta os órfãos; o DELETE passou a verificar se a pasta existe , não se o campo está preenchido
Lançamento órfão (encontrado ao limpar dados de teste)	Excluir a pasta deixava os <code>prontuario_registros</code> órfãos no banco — registro clínico invisível mas presente — e anamnese e agenda apontando para um número inexistente	<code>vinculosDe("prontuario")</code> passou a contar lançamentos, anamneses e agendamentos; o DELETE devolve 409 sugerindo Dar alta

RECOMENDAÇÃO

Declarar as chaves estrangeiras, com `ON DELETE RESTRICT` onde a aplicação já recusa e `ON DELETE SET NULL` nos vínculos opcionais (`atendimentos.prontuario_id`, `anamneses.prontuario_id`). O trabalho é uma migration nova e um saneamento prévio dos órfãos existentes. O ganho: o banco deixa de aceitar o estado que já produziu dois defeitos, e as rotinas de remendo passam a ser redundância em vez de única defesa. **É a recomendação de maior impacto deste documento.**

7. Migrations

Cada arquivo `.sql` em `migrations/` roda **uma vez**, na ordem do nome, dentro de **uma transação**, e fica registrado em `schema_migrations`. Migração que falha **para o boot**.

Arquivo	O que faz	Por que existe
001_esquema_inicial.sql	Cria as 13 tabelas do sistema, 6 índices de desempenho e os 3 índices únicos que gravam as regras de negócio	Tradução fiel do que existia no SQLite, já com todas as colunas que antes eram acrescentadas por <code>ALTER TABLE</code> no boot
002_auditoria.sql	Cria a tabela auditoria e 4 índices	Responder “quem mexeu nisto, e quando” — pergunta que, num sistema de prontuário, uma hora aparece
003_dono_do_prontuario.sql	Acrescenta <code>profissional_id</code> a <code>prontuario</code> e anamneses, preenche os registros antigos casando pelo nome, e cria 2 índices	Regra nova: o profissional vê apenas os prontuários e as anamneses dele. Faltava uma ligação verificável entre o registro e o profissional
004_arquivar_pasta_e_paciente.sql	Acrescenta <code>arquivado</code> e <code>arquivado_em</code> a <code>pacientes</code> e <code>prontuario</code> , cria 2 índices parciais e dois <code>COMMENT ON COLUMN</code>	Separar “arquivar” de “inativar” e de “dar alta”: três conceitos diferentes, cuja mistura destruiria informação de trabalho

7.1 Qualidade das migrations

- Todas são **idempotentes**: usam `IF NOT EXISTS` e `ADD COLUMN IF NOT EXISTS`, e podem rodar de novo sem efeito.
- Todas trazem, no cabeçalho, o **raciocínio da mudança** — não apenas o comando. A 003 explica por que `profissional` (texto) e `usuario_id` não serviam; a 004 explica por que “arquivar” não podia reaproveitar “ativo”.
- A 003 e a 004 **preenchem os dados existentes** na mesma transação da mudança de estrutura, o que evita o estado intermediário em que a coluna existe vazia.
- A 004 usa `COMMENT ON COLUMN` para deixar a distinção **dentro do próprio banco**, ao alcance de quem abrir só o esquema.

FATO

A migration 003 escolhe deliberadamente o **lado seguro do erro**: o que não casou pelo nome ficou com dono nulo, e registro sem dono nenhum profissional vê. O comentário justifica: “esconder demais se conserta atribuindo a pasta; mostrar de menos não expõe prontuário de ninguém”.

PONTO DE ATENÇÃO - conferência recomendada

`migrar.js --status` avisa quando existe migração **registrada cujo arquivo sumiu** — sinal de que alguém apagou um `.sql` já aplicado, e o banco passou a ter uma alteração que o código não sabe mais descrever. Vale rodar esse comando em toda auditoria de ambiente.

8. Dados iniciais (seeds)

Semeados na primeira inicialização, sempre condicionados a `COUNT(*) = 0` ou a uma trava em `g_config` — nunca sobrescrevem escolha da clínica.

Tabela	Quantidade	Conteúdo	Condição
<code>g_usuarios</code>	1	Conta admin inicial, com senha padrão que deve ser trocada no primeiro acesso	Nenhum administrador ativo
<code>convenios</code>	8	Particular · Cartão BemEstarClinic · Efycard · Forms Fitness Academia · Pad Saúde · Prosmed · São Gabriel · System Saúde	Tabela vazia
<code>salas</code>	3	Consultório 01 · Consultório 02 · Consultório 03	Tabela vazia
<code>procedimentos</code>	19	11 tratamentos, desdobrados em Consulta, Sessão e Procedimento — todos com duração de 40 minutos	Tabela vazia

8.1 Cores oficiais dos procedimentos

As cores não foram escolhidas por estética: foram **extraídas do documento de agenda do cliente**, onde cada procedimento estava escrito na sua cor. A cor é por **família** — consulta e sessão do mesmo tratamento usam a mesma.

Procedimento	Cor	Procedimento	Cor
Psicanálise Individual	#FF0000	Terapia Floral	#FF0066
Psicanálise Casal	#0000CC	Biorressonância	#00FF00
Protocolo Integrativo	#4472C4	Ventosaterapia	#FFFF00
Ozonioterapia	#538135	Kinesioterapia	#00CCFF
Detox lônico	#FFC000	Acupuntura	#C00000
Aromaterapia	#7030A0	(padrão, se não mapeado)	#5B4FD8

FATO

A aplicação das cores oficiais é protegida pela trava `g_config.cores_oficiais = '1'`: ela roda **uma única vez**. Se a clínica trocar uma cor depois, o deploy seguinte não desfaz a escolha dela. O mesmo padrão de trava protege a sementeira do modelo de anamnese por procedimento (`anamnese_modelo_seed`).

INFERÊNCIA

Na interface, a cor é exibida como **etiqueta preenchida** e não como cor de texto, com a cor da letra decidida por luminância. Duas das cores oficiais — verde-limão (`#00FF00`) e ciano (`#00CCFF`) — são claras demais para texto sobre fundo branco, o que justifica a escolha.

8.2 Sequências de numeração

Os contadores vivem em `g_config`, como `pront_seq_<ano>` e `pac_seq_<ano>`. Eles **só sobem**, e a emissão usa `max(contador, MAX no banco) + 1`.

FATO

Guardar o contador fora da tabela resolve um defeito que existiu: gerar o próximo número por `MAX(numero)` **reaproveitava** o número de um registro excluído, porque o maior caía. Ler também o `MAX` do banco é a rede de segurança para o caso de o `g_config` se perder.

9. Proteção dos dados

9.1 Colunas cifradas

Os campos abaixo são gravados **cifrados** (AES-256-GCM, aplicado pela aplicação) e voltam decifrados na leitura. Na tela e na impressão nada muda; no banco, num dump ou num backup vazado, não há nada legível.

Tabela	Colunas cifradas	Qtd.
pacientes	cpf · rg · nascimento · naturalidade · estado_civil · religiao · profissao · escolaridade · altura · peso · cor_pele · sangue · cep · endereco · numero · bairro · cidade · complemento · celular · telefone · email · canal · mae · pai · indicacao · nome_contato · foto · observacao · inativo_motivo · resp_nome · resp_cpf · resp_rg · resp_nascimento	33
anamneses	dados (todas as respostas)	1
prontuario_registros	texto · anexo	2
prontuario	observacao · alta_motivo	2
historico	detalhe (guarda trechos do que foi editado)	1
atendimentos	nome_agenda · celular · observacoes	3
documentos_gestao	titulo · arquivo	2
profissionais	contato · registro	2
auditoria	resumo · detalhe	2

48 colunas cifradas em 9 tabelas.

9.2 O que NÃO é cifrado — e por quê

Fica legível	Motivo declarado
pacientes.nome e pacientes.codigo	São as chaves de busca e ordenação das listas. Cifrados, o banco não conseguiria ordenar por nome nem procurar por parte dele, e cada listagem traria a clínica inteira para a memória antes de mostrar a primeira linha
Datas usadas em filtro de período	Precisam ser comparadas no SQL
status, ids e os liga/desliga	Idem
Colunas de filtro da auditoria (criado, ip, usuario_nome, perfil, acao, modulo)	É por elas que a tela filtra e agrupa — sem revelar paciente

INFERÊNCIA

O argumento registrado no código para deixar o nome legível é que “um nome sozinho, sem CPF, sem endereço, sem telefone e sem prontuário, é o que já aparece na agenda impressa em cima do balcão”. O raciocínio é defensável, **mas tem uma consequência que precisa ser dita**: um dump vazado revela **quem são os pacientes de uma clínica de saúde mental**. A simples associação nome + clínica já é dado sensível em contexto psiquiátrico e psicológico, mesmo sem nenhum outro campo.

RECOMENDAÇÃO

Registrar essa avaliação formalmente no relatório de impacto (RIPD) da clínica, com a justificativa de necessidade — a decisão é razoável e tem contrapartida técnica concreta, mas deve estar documentada como escolha consciente, não como omissão.

9.3 Busca sobre campo cifrado

Um valor cifrado com IV aleatório não pode ser comparado no SQL: o mesmo CPF gera texto diferente a cada gravação. O sistema resolve isso de duas formas:

Necessidade	Solução	Limite
Achar o paciente por este CPF exato	Impressão digital: HMAC-SHA256 do valor normalizado, guardado ao lado. HMAC e não hash puro — um SHA256 de CPF é reversível na prática, são 11 dígitos	Serve para igualdade, não para busca parcial
Busca por parte do CPF (o que a recepção usa)	A aplicação decifra e filtra em memória , depois de trazer as linhas do banco	A lista de pacientes não pagina no banco — quem pagina é a tela. O custo cresce com o tamanho do cadastro

Mínimo de **3 dígitos** para a busca por CPF, no servidor e no seletor: com um dígito, qualquer CPF casaria e a busca devolveria a clínica inteira.

9.4 Backup e restauração

Aspecto	Como é
Método	pg_dump com <code>--clean --if-exists</code> — restaurar substitui o banco inteiro, não mistura
Frequência	Diária, dentro do próprio processo, sem cron. Se a máquina estava desligada na hora marcada, a cópia sai no próximo boot
Retenção	30 cópias, em backups/
Sob demanda	Botão Backup do banco no menu do /restrito (só admin). O arquivo é transmitido direto ao navegador , nunca gravado num diretório servido pela web
Conteúdo	Os dados sensíveis saem cifrados no dump — legíveis apenas em servidor com a mesma chave
Restauração	<code>restaurar.sh</code> confere a integridade antes de sobrescrever, guarda o estado atual e exige confirmação digitada

PONTO DE ATENÇÃO - risco de perda total

Um backup sem a chave não serve para restaurar. A cifragem que protege o dump vazado é a mesma que o inutiliza se a DADOS_CHAVE se perder. Chave e backup precisam ser guardados — em lugares separados, mas **ambos**. Não há, no repositório, procedimento documentado de custódia dessa chave.

9.5 Superfície de exposição do banco

- PostgreSQL escuta **apenas em 127.0.0.1**; a porta 5432 não é aberta no firewall.
- /data, /backups e as extensões .db, .sql e .sqlite respondem **404** na aplicação.
- Todo SQL usa **consultas parametrizadas**; nome de tabela e coluna vêm de whitelists (TAB, TABLES) e das colunas reais lidas do catálogo.
- Datas de filtro são validadas por `^\d{4}-\d{2}-\d{2}$` antes de entrar na consulta.
- A credencial do banco vem do ambiente do serviço, nunca do repositório.

FATO

Um teste de intrusão registrado no projeto reportou SQL injection porque o payload `1 UNION SELECT * FROM g_usuarios` devolveu uma linha. **Era falso positivo:** nada foi injetado — a busca por CPF sem máscara casava com qualquer dígito, e o CPF do paciente continha “1”. Ainda assim virou correção real, com o mínimo de 3 dígitos passando a valer também no servidor.

10. Consultas relevantes

Consultas cuja lógica ajuda a entender o sistema. Reproduzidas de forma resumida.

Validação de choque na agenda

Antes de gravar um atendimento, o servidor verifica **duas** sobreposições independentes — a do profissional e a da sala — no mesmo dia, ignorando os cancelados e o próprio registro em edição. É a consulta que sustenta a regra operacional mais visível do sistema.

Recolhimento retroativo de agendamentos

Ao finalizar uma anamnese, `recolherAtendimentosSoltos()` procura os atendimentos daquele **paciente** com aquele **procedimento** que ainda estão com `prontuario_id` nulo, e os vincula à pasta recém-criada. É o que faz o histórico ficar completo mesmo quando o paciente já vinha sendo atendido antes da anamnese.

Prontuário completo

`GET historico/:pacienteId` monta, em ordem cronológica, as anamneses, as pastas com seus lançamentos e os atendimentos. Para o perfil **profissional**, a consulta acrescenta dois recortes: só as pastas dele e só os lançamentos que ele mesmo escreveu.

FATO

O recorte precisa existir **nessa consulta também**, e não apenas nas listagens: é ela que alimenta a impressão do prontuário completo. Se falhasse aqui, o profissional imprimiria o acompanhamento de pacientes que não são dele — pela via mais difícil de auditar, que é o papel.

Relação de pacientes ativos e inativos

`GET relatorios/pacientes?ativo=1|0` reúne, por paciente, endereço completo, WhatsApp, quem o assiste e as especialidades. Duas decisões de dado sustentam esse relatório:

- “**Quem assiste**” **soma duas origens**: o profissional responsável de cada prontuário e quem de fato atendeu na agenda. Podem ser pessoas diferentes, e para uma relação de contato as duas importam. O resultado é deduplicado.
- **A especialidade vem das pastas e dos procedimentos agendados** — senão quem ainda não tem prontuário aberto sairia com a coluna vazia.

Contagem de vínculos antes de excluir

`vinculosDe(tabela, id)` faz um `COUNT(*)` por relação e monta a frase que o usuário lê (“1 atendimento, 1 anamnese...”). Essa é a consulta que depende do analisador de tipo `bigint` descrito na documentação técnica: sem ele, `COUNT` voltaria como a string “0”, que é verdadeira em JavaScript, e o sistema recusaria excluir cadastros sem nenhum vínculo.

11. Observações e recomendações

Separação explícita entre o que foi encontrado e o que se recomenda.

11.1 Problemas encontrados

#	Achado	Onde	Gravidade
1	Nenhuma FOREIGN KEY declarada. As 14 relações são apenas colunas mantidas pela aplicação. Já produziu dois defeitos reais de dado órfão	Todo o esquema da gestão	Alta
2	Nenhuma CHECK constraint. Domínios fechados (perfil, status, tipo) só são validados na aplicação	Todo o esquema da gestão	Média
3	ALTER TABLE em try/catch vazio — o padrão que a gestão abandonou continua vivo no banco do site, para 5 colunas	server.js, banco do site	Média
4	Nome de coluna enganoso. prontuario.especialidade guarda o nome do <i>procedimento</i> . Documentado no esquema, mas o nome permanece	prontuario	Baixa
5	Relação N:N resolvida por texto. profissionais.especialidade guarda rótulos, não ids: renomear um procedimento quebra a associação em silêncio	profissionais	Média
6	Colunas sem consumidor. atendimentos.lembrete e atendimentos.nps existem e são preenchidas, mas nenhuma rotina as lê	atendimentos	Baixa
7	Banco legado no disco. O data/gestao.db (155 KB) continua no ambiente analisado. O POSTGRES.md instrui renomeá-lo para .antes-do-postgres	data/	Baixa
8	Datas como TEXT. Impede aritmética de data no SQL e validação de formato pelo próprio banco. É decisão consciente e documentada	Todo o esquema	Baixa
9	Listagem de pacientes sem paginação no banco. Consequência da cifragem do CPF: todas as linhas são trazidas para filtrar em memória	pacientes	Baixa hoje
10	Backup apenas local. As cópias ficam na mesma máquina do banco	backups/	Alta

NÃO IDENTIFICADO

Não foram encontrados: campos redundantes entre tabelas, dados duplicados, estruturas obsoletas remanescentes, nem problemas de normalização. O modelo está adequadamente normalizado para o domínio, e a nomenclatura é consistente — em português, com snake_case e sufixo _id para vínculos. As duas exceções de nomenclatura (especialidade e o prefixo g_ nas tabelas de sistema) estão documentadas e são coerentes dentro de si.

11.2 Recomendações

Prioridade	Recomendação	Esforço	Resolve
1	Declarar as chaves estrangeiras , com RESTRICT onde a aplicação já recusa e SET NULL nos vínculos opcionais. Exige sanear os órfãos antes	Uma migration + saneamento	Achado 1
2	Destino de backup externo, com criptografia em trânsito e em repouso; e ensaiar a restauração no servidor real	Médio	Achado 10
3	Documentar a custódia da DADOS_CHAVE: onde fica a cópia, quem tem acesso, como se recupera. Hoje perdê-la torna os backups inúteis	Baixo	Risco 9.4
4	Acrescentar CHECK nos domínios fechados (perfil, status, tipo). Custo baixo, ganho imediato de garantia	Uma migration	Achado 2
5	Migrar os 5 ALTER TABLE do banco do site para um mecanismo com registro, ainda que simples	Baixo	Achado 3
6	Trocar profissionais.especialidade por tabela de junção com procedimento_id	Médio	Achado 5

Prioridade	Recomendação	Esforço	Resolve
7	Decidir sobre lembrete e nps: implementar o consumo ou remover as colunas. Coluna que não faz nada engana quem lê o esquema	Baixo	Achado 6
8	Renomear prontuario.especialidade para procedimento_nome, com migration e ajuste da aplicação	Médio	Achado 4
9	Arquivar ou remover o gestao.db legado, conforme o procedimento já documentado	Trivial	Achado 7

FATO

Nota de método. Toda tabela, coluna, índice e restrição descrita aqui foi lida nos arquivos de migration e no código que cria o banco do site. Nenhuma estrutura foi inferida. Nenhum dado real de paciente, credencial ou chave foi consultado ou reproduzido: a análise foi feita sobre o **esquema**, não sobre o conteúdo dos bancos.